



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)
Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltes.com Email: principal@dypatiltes.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil

Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil

Chairman

Adv. Anuja Sushant Patil

Director

Dr. Suresh Shirbahadurkar

Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: Python Language

Prepared By: Mansi Ghodke

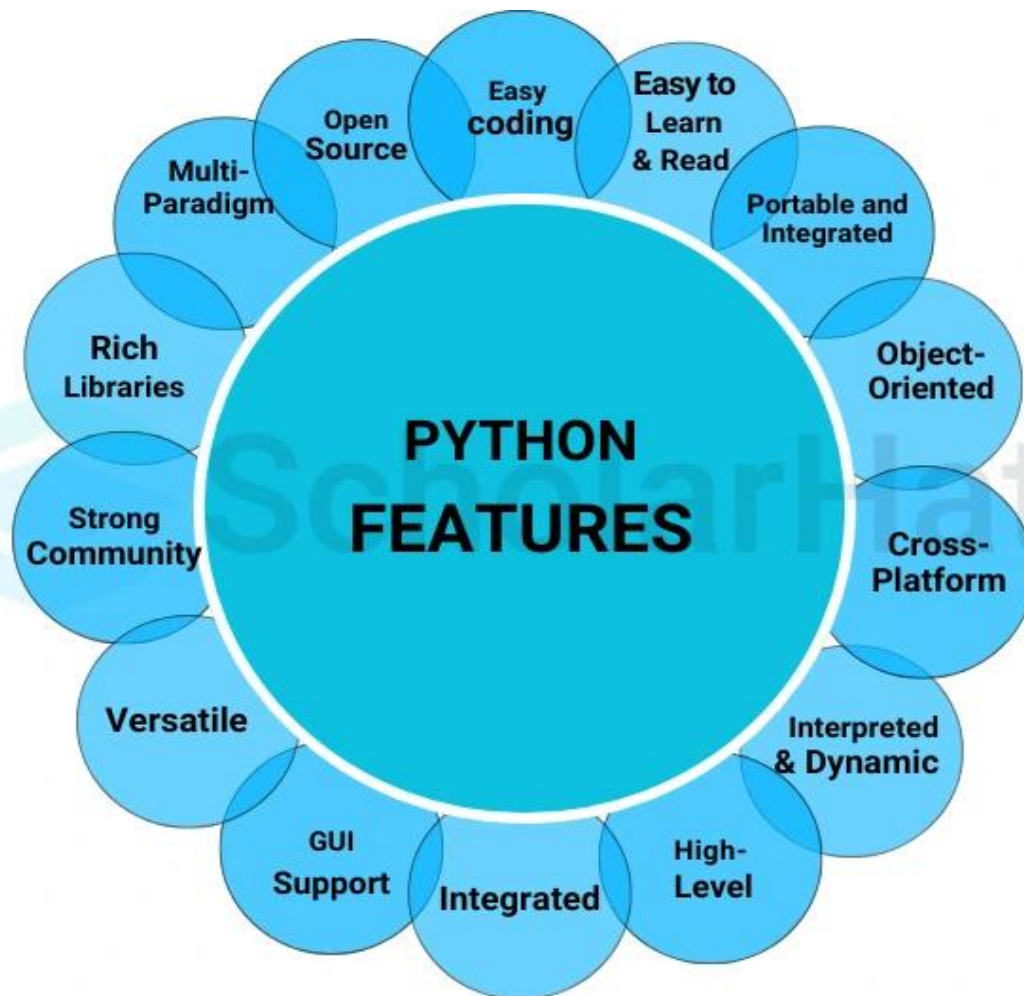
Python Programing Language

What is Python?

Python is a **high-level, general-purpose programming language** known for readable syntax and a large standard library.

Python is a beginner-friendly yet powerful programming language, widely used in data science, AI, web development, and automation. Its core concepts include variables, data types, control flow, functions, data structures, object-oriented programming, exception handling, and file/database management. Mastering these fundamentals gives you the foundation to build anything from small scripts to large-scale applications.

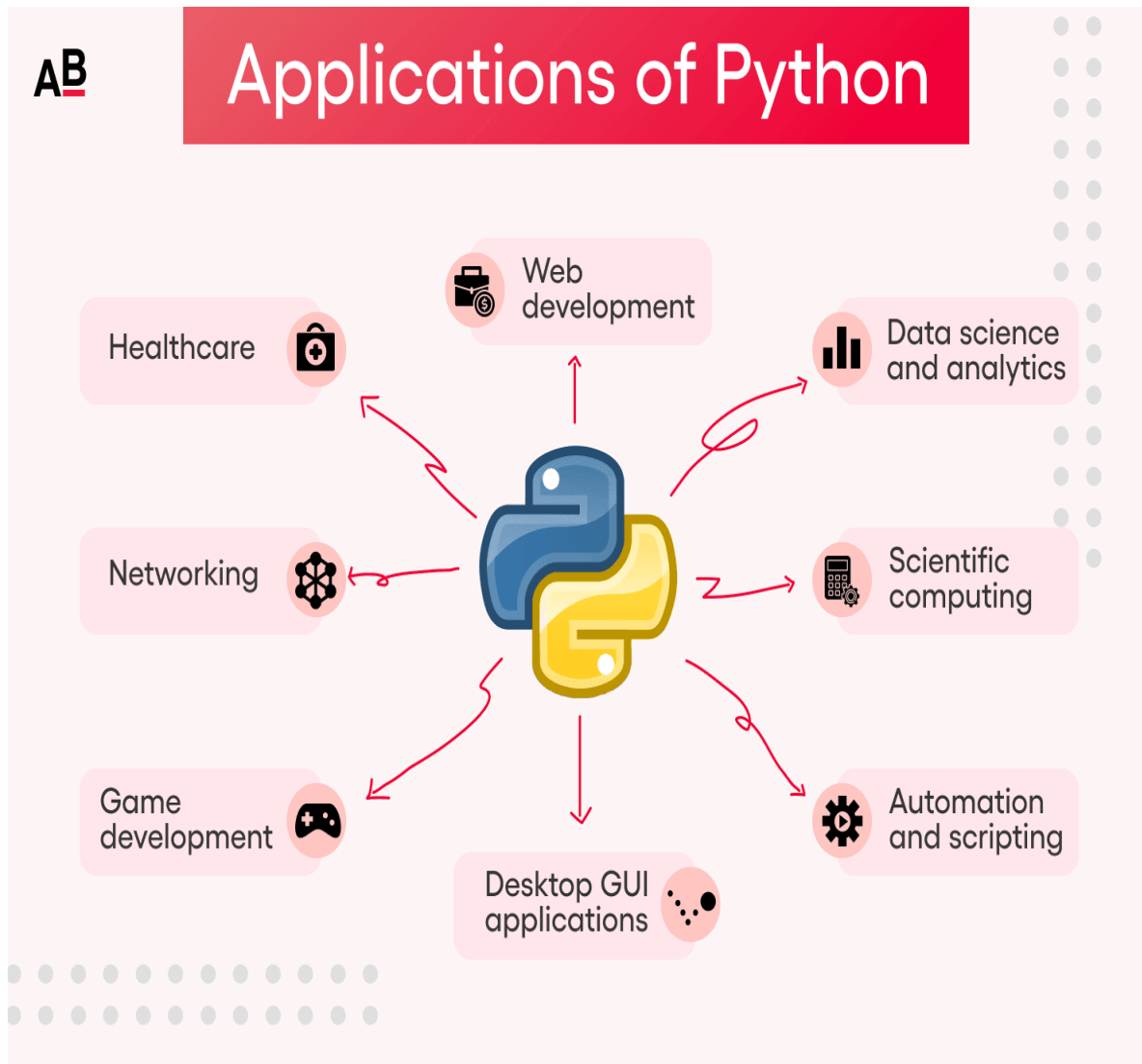
✓ Features Of Python



Key Features of Python

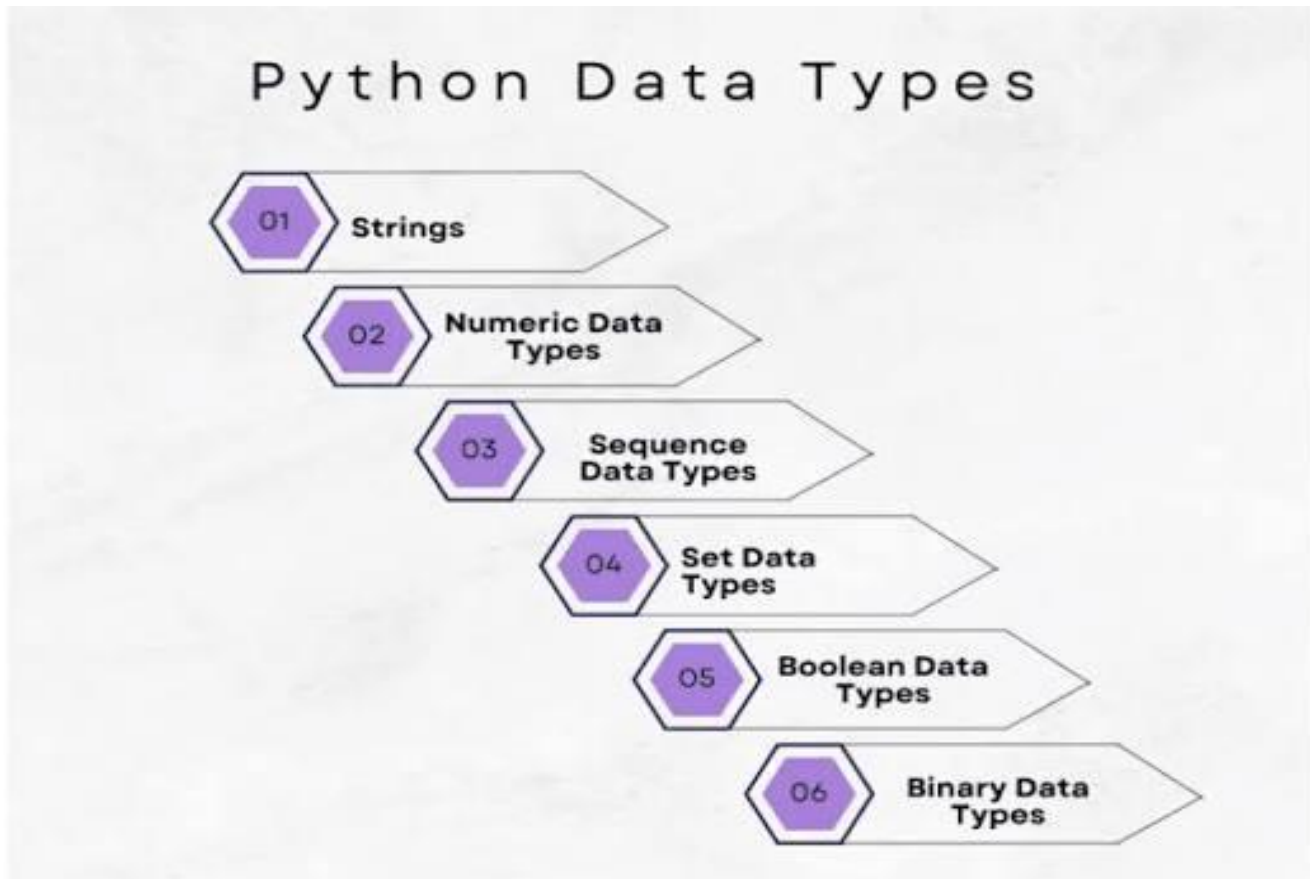
- **Interpreted language:** No compilation step; code runs directly, speeding up development.
- **Object-oriented:** Supports classes, inheritance, and encapsulation.
- **Dynamic typing:** Variables don't need explicit type declarations.
- **Extensive standard library:** Includes modules for file I/O, networking, math, and more.
- **Cross-platform:** Runs on Windows, macOS, Linux, and more.
- **Community support:** Thousands of third-party packages available via PyPI.

✓ Application Of Python



Python is an interpreted, high-level, dynamically typed, and object-oriented programming language renowned for its clean syntax, code readability, and vast library ecosystem. Because it removes compilation steps and enforces indentation instead of curly braces, developers can express complex program design choices with fewer lines of code. python is a high-level, interpreted, and multi-paradigm programming language renowned for its clear readability and clean, intuitive syntax. Because it relies on **indentation instead of curly braces** to mark blocks of code, Python naturally forces developers to write clean, visually organized programs

✓ Python Data Types



✓ Control Flow & Structure

Python programs evaluate conditions and loop through sequences using readable English keywords. [\[1\]](#)

- **Conditional Statements (if-elif-else):** Directs the execution flow based on true/false evaluation.
- **Loops:** for loops iterate over a known collection or range, while while loops run continuously as long as a baseline condition remains true.
- **List Comprehensions:** A clean, single-line alternative for creating new lists out of existing collections.

✓ Functions & Scope

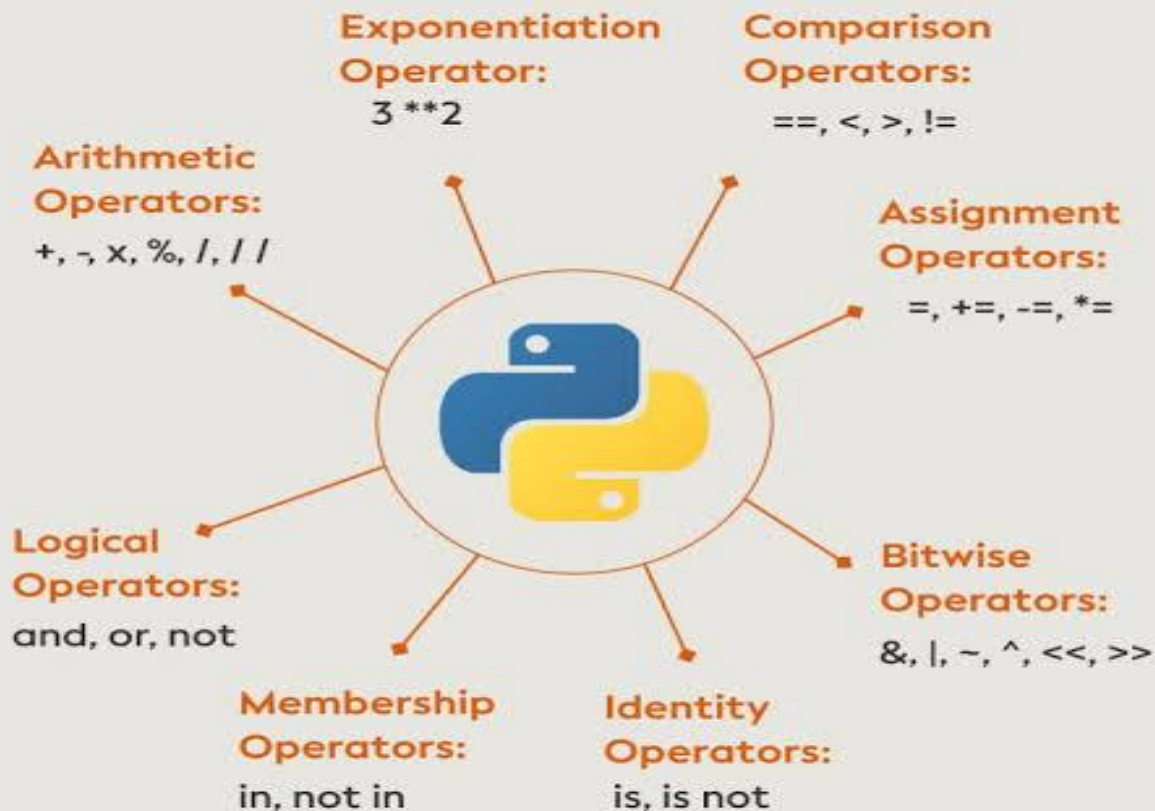
Functions encapsulate reusable blocks of code to prevent repetition.

- **def Keyword:** Declares standard functions.
- **lambda Functions:** Small, anonymous, single-expression functions used for quick computations.
- **Variable Scope:** Variables defined inside a function are *local* to that function. If defined outside, they belong to the *global* scope.

✓ Python Operators



PYTHON OPERATORS



Techie Programmer

©2024

1.Arithmetic Operators

Used to perform standard mathematical calculations. [[1](#), [2](#)]

- **+** **Addition:** $5 + 3 \rightarrow 8$
- **-** **Subtraction:** $5 - 3 \rightarrow 2$
- ***** **Multiplication:** $5 * 3 \rightarrow 15$
- **/** **Division:** Returns a float ($5 / 2 \rightarrow 2.5$)
- **//** **Floor Division:** Rounds down to the nearest integer ($5 // 2 \rightarrow 2$)
- **%** **Modulus:** Returns the remainder ($5 \% 2 \rightarrow 1$)
- ****** **Exponentiation:** Power operator ($5 ** 3 \rightarrow 125$)

2. Assignment Operators

Used to assign or update values to variables. [[1](#), [2](#)]

- **= Assign:** $x = 5$
- **+= Add and Assign:** $x += 3$ (Equivalent to $x = x + 3$)
- **-= Subtract and Assign:** $x -= 3$
- ***= /= / %= Compound Assigns:** Applies the math operation and updates the variable.

3. Comparison (Relational) Operators

Compare two values and always return a Boolean: True or False.

- **== Equal to:** $5 == 3 \rightarrow \text{False}$
- **!= Not equal to:** $5 != 3 \rightarrow \text{True}$
- **> / < Greater / Less than:** $5 > 3 \rightarrow \text{True}$
- **>= / <= Greater / Less than or equal to**

4. Logical Operators

Used to combine conditional statements. [[1](#)]

- **and Logical AND:** Returns True if both statements are true ($x > 2$ and $x < 10$)
- **or Logical OR:** Returns True if at least one statement is true
- **not Logical NOT:** Reverses the result ($\text{not}(\text{True}) \rightarrow \text{False}$)

5. Membership Operators

Used to test if a sequence (like a string, list, or tuple) contains a specific value.

- **in** Returns True if the value is found in the sequence.
- **not in** Returns True if the value is missing from the sequence.

6. Bitwise Operators

Used to perform bit-by-bit operations on binary numbers. [[1](#), [2](#)]

- **& AND | | OR | ^ XOR | ~ NOT | << Left Shift | >> Right**

✓ Concept: Operator Precedence

When multiple operators appear in a single expression (e.g., $x = 5 + 3 * 2$), Python uses **Operator Precedence** to decide what to calculate first.

- **Parentheses ()** have the highest precedence.
- **Math operations** (like $*$ and $/$) happen before addition/subtraction ($+$, $-$).
- **Comparison operators** happen before **Logical operators**. .

Python Concepts Roadmap

Beginner

1. Introduction to Python
 - Overview of Python
 - Installing Python and an IDE
 - Basic syntax and data types
2. Control Flow and Functions
 - Conditional statements (if, else, elif)
 - Loops (for, while)
 - Functions and modular programming
3. Data Structures
 - Lists, tuples, and dictionaries
 - String manipulation
 - Basic input/output
4. File Handling
 - Reading and writing files
 - Working with different file formats (text, CSV)
5. Exception Handling
 - Handling errors and exceptions
 - Using try-except blocks
6. Introduction to OOP (Object-Oriented Programming)
 - Classes and objects
 - Inheritance and polymorphism
7. Introduction to Libraries
 - Overview of standard libraries (e.g., math, random)
 - Introduction to external libraries (e.g., NumPy for numerical operations)

INTERMEDIATE

1. Advanced OOP
 - Encapsulation, abstraction, and inheritance
 - Advanced topics in polymorphism
2. Advanced Data Structures
 - Sets and frozensets
 - Stacks and queues
 - Advanced list comprehensions
3. Functional Programming
 - Lambda functions
 - Map, filter, and reduce functions
 - Decorators and closures
3. Functional Programming
 - Lambda functions
 - Map, filter, and reduce functions
 - Decorators and closures
4. Working with APIs
 - Making API requests (e.g., using requests library)
 - Parsing JSON responses
5. Database Interaction
 - Basics of database connectivity (e.g., SQLite)
 - CRUD operations with databases
6. Introduction to OOP (Object-Oriented Programming)
 - Classes and objects
 - Inheritance and polymorphism
7. Introduction to Libraries
 - Overview of standard libraries (e.g., math, random)
 - Introduction to external libraries (e.g., NumPy for numerical operations)

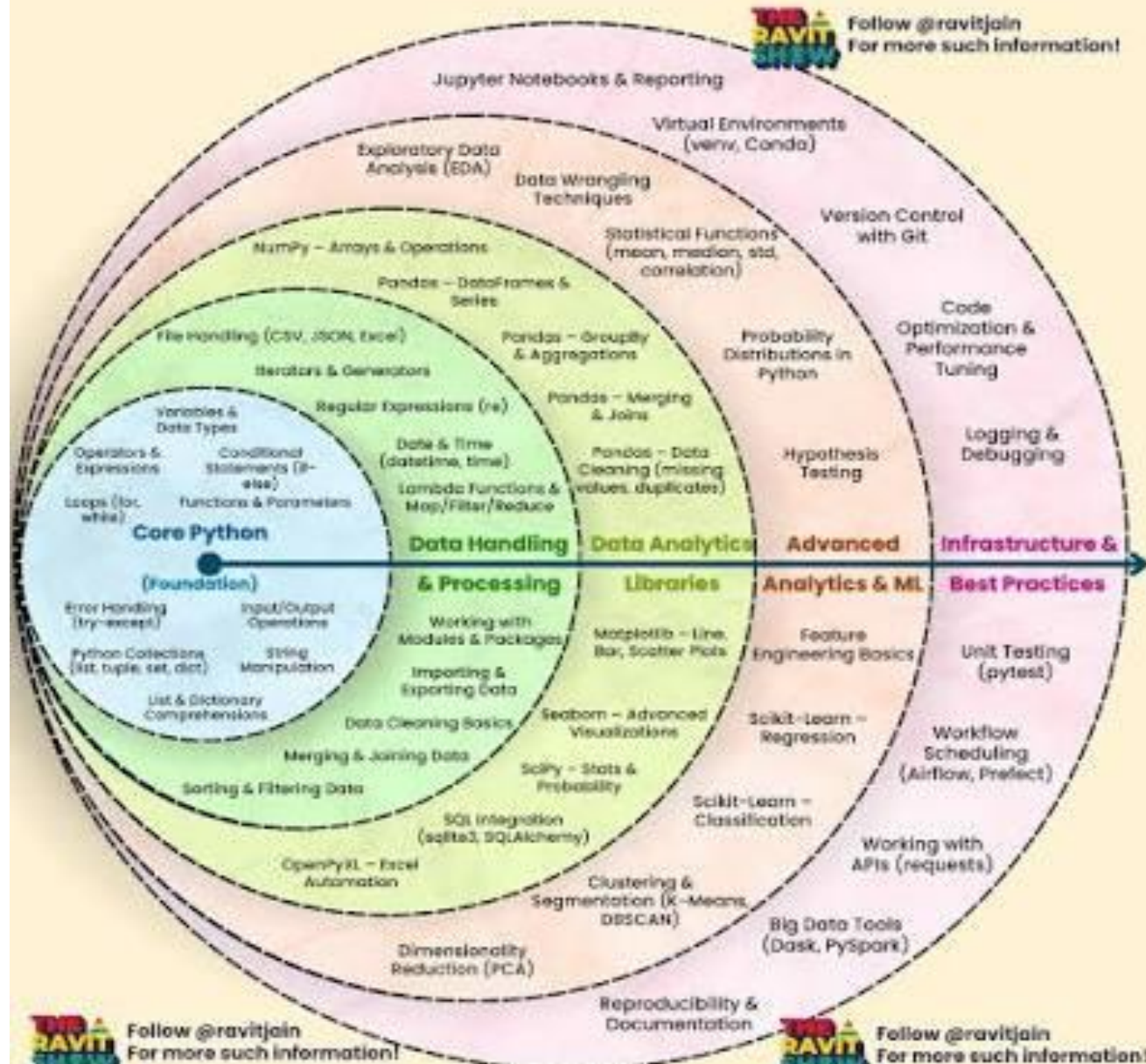
ADVANCED

1. Advanced Web Development
 - RESTful APIs
 - Authentication and authorization
 - Front-end frameworks (e.g., React, Angular)
2. Concurrency and Parallelism
 - Threading and multiprocessing
 - Asyncio for asynchronous programming
3. Advanced Libraries and Frameworks
 - Deep dive into popular libraries (e.g., Pandas, NumPy, TensorFlow)
 - Customizing and extending frameworks
4. Security in Python
 - Best practices for secure coding
 - Handling common security vulnerabilities
5. Optimization and Performance
 - Profiling and optimizing code
 - Memory management and optimization techniques
6. Advanced Topics in Data Science
 - Machine learning with Python (e.g., scikit-learn)
 - Data Visualization with Matplotlib and Seaborn
7. Contributing to Open Source Projects
 - Understanding open source contribution workflows
 - Collaborating on GitHub and other version control system

Python Concepts For Data Analytics



Follow @ravitjain
For more such information!



Follow @ravitjain
For more such information!



Follow @ravitjain
For more such information!

✓ Variables, Memory, and Dynamic Typing

dynamic typing, meaning you do not declare variable types explicitly. The interpreter determines the type at runtime based on the value assigned.

- **Object References:** In Python, variables are labels pointing to objects stored in memory, not the memory slots themselves.
- **Mutability:** Objects are either **mutable** (can be changed in place) or **immutable** (cannot be changed after creation).

```
# Dynamic typing
x = 10      # x is an int
x = "Hello" # x is now a str (perfectly valid)
```

```
# Mutability Demonstration
list_a = [1, 2, 3]
list_b = list_a    # Both point to the same memory object
list_a.append(4)
print(list_b)      # Output: [1, 2, 3, 4] (Lists are mutable)
```

```
num_a = 5
num_b = num_a
num_a += 1
print(num_b)
```

Output: 5 (Integers are immutable; num_a created a new object)

✓ Built-in Data Structures Deep Dive

Python's built-in collections have unique performance characteristics and use cases:

- **Lists ([])**: Ordered, mutable sequences. Excellent for linear collections where items change frequently.
- **Tuples (())**: Ordered, immutable sequences. Faster than lists and protect data from accidental modification. Frequently used to return multiple values from functions.
- **Dictionaries ({k: v})**: Hash tables that store key-value pairs. Lookup, insertion, and deletion times are incredibly fast— $O(1)$ on average.

```
# Dictionary lookup and Set uniqueness
```

```
user_roles = {"alice": "admin", "bob": "editor"}  
print(user_roles["alice"])
```

```
# Output: admin
```

```
unique_ids = {101, 102, 101, 103}  
print(unique_ids)      # Output: {101, 102, 103} (Duplicates removed)
```

✓ Advanced Control Flow & Iterables

Beyond standard if statements, Python uses powerful syntax to process sequences efficiently.

- **List Comprehensions:** A declarative, highly readable way to generate new lists out of existing iterables. They execute faster than standard for loops because they run at C-level speeds internally.
- **Generators:** Functions that use the yield keyword instead of return. They produce items lazily (one at a time), saving massive amounts of RAM when dealing with massive datasets.

```
# List Comprehension: Square even numbers
```

```
numbers = [1, 2, 3, 4, 5, 6]  
squares_of_evens = [x**2 for x in numbers if x % 2 == 0]  
print(squares_of_evens) # Output: [4, 16, 36]
```

```
# Generator Function
```

```
def count_up_to(max_val):  
    count = 1  
    while count <= max_val:  
        yield count  
        count += 1
```

```
counter = count_up_to(1000000) # Evaluates lazily; consumes almost zero memory  
print(next(counter))          # Output: 1  
print(next(counter))          # Output: 2
```

✓ Object-Oriented Programming (OOP)

Python is a multi-paradigm language that fully supports **Object-Oriented Programming** to model real-world scenarios using classes and objects.

- **Classes & Objects:** A Class is a blueprint; an Object is an instance of that blueprint.
- **Encapsulation:** Restricting access to methods and variables by using a single underscore _ (protected) or double underscore __ (private/name-mangled).
- **Inheritance:** Creating a child class that inherits attributes and methods from a parent class.
- **Polymorphism:** Allowing different classes to use methods with the identical name but distinct logic.

```
class Animal:
    def __init__(self, name):
        self.name = name # Instance variable

    def speak(self):
        raise NotImplementedError("Subclass must implement this method")

class Dog(Animal):
    def speak(self):
        return f'{self.name} says Woof!'

class Cat(Animal):
    def speak(self):
        return f'{self.name} says Meow!'

# Polymorphism in action
pets = [Dog("Buddy"), Cat("Whiskers")]
for pet in pets:
    print(pet.speak())

# Output:
# Buddy says Woof!
# Whiskers says Meow!
```



Important Concepts in Python Programming



Basics

- Basics syntax
- Variables
- Data types
- Conditionals
- Typecasting
- Exceptions
- Functions
- Lists, Tuples, Sets
- Dictionaries

Advanced

- List
- Comprehensions
- Generators
- Expressions
- Paradigms
- Regex
- Decorators
- Iterators
- Lambdas

OOP

- Classes
- Inheritance
- Methods

Data Science

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- TensorFlow
- PyTorch

Data Structures and Algorithms

- Arrays and Linked Lists
- Heaps, Stacks, Queues
- Hash Tables
- Binary Search Trees
- Recursion
- Sorting Algorithms

Web Frameworks

- Django
- Flask
- FastAPI
- Tornado

Package Manager

- PyPI
- pip
- conda

Automation

- File manipulation
- Web scraping
- GUI automation
- Network automation

